SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION with Open CV COURSE CODE: DSA0204

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison.* (BTL 4)

Assessment Tool Weightage: 20%

Read the given scenario carefully and analyze the problem using appropriate computer vision techniques. Justify your answers with relevant reasoning and comparisons wherever necessary.

Code of Conduct:

I **C. Sai Harshitha (192424346)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: C. Sai Harshitha

1. **Hough Transform for Shape Detection**

   A system uses Hough Transform to detect circular shapes but fails in complex backgrounds. Analyze the limitations and explain how preprocessing or parameter tuning can improve detection.

The Hough Transform is commonly used to detect geometric shapes such as circles by identifying edge points that satisfy a particular geometric relationship. In this case, the system fails because a complex background contains many unwanted edges, textures, and patterns that can interfere with circle detection.

Analysis of the Problem

The main limitation is that the Hough Transform does not inherently know which edges belong to the desired circular object and which belong to the background. As a result, background edges can produce incorrect circle candidates.

This can lead to:

- **False positives**, where background patterns are detected as circles.
- **Missed circles**, when the actual circular boundary is weak or interrupted.
- **Multiple detections** of the same circle.
- Increased computational cost because a large number of edge points must be processed.

Therefore, the complex background reduces the reliability of the detected shape.

How Preprocessing Improves Detection

Preprocessing can improve the input image before applying the Hough Transform.

1. Noise reduction:

Gaussian or median filtering can reduce small unwanted variations and prevent noise from being interpreted as circular edges.

2. Grayscale conversion:

Converting the image to grayscale simplifies the image and allows edge information to be processed based on intensity.

3. Edge detection:

Applying an edge detector such as Canny can produce a cleaner set of edges for the Hough Transform.

4. Contrast enhancement:

If the circle has weak contrast with its surroundings, contrast enhancement can make its boundary more prominent.

5. Region of Interest (ROI):

If the approximate location of the circular object is known, restricting detection to that region can eliminate irrelevant background areas.

Parameter Tuning

Proper selection of Hough Transform parameters is also important.

- **Minimum and maximum radius:** Restricts detection to circles of the expected size.
- **Detection threshold:** A higher threshold can reduce false detections, while a lower threshold can help detect weak circles.
- **Minimum distance between circle centers:** Prevents several detections of the same circle.
- **Accumulator resolution:** Controls the precision of the parameter-space search and can affect detection quality and computation time.

Justification

Using preprocessing alone may not completely solve the problem because some background structures may still produce strong edges. Similarly, parameter tuning alone may fail if the input contains excessive noise. Therefore, combining preprocessing with appropriate Hough parameters is more effective.

For example, noise reduction followed by Canny edge detection can provide cleaner edges, while restricting the radius range prevents the algorithm from considering unrealistic circles. ROI selection can further eliminate irrelevant background regions.

Conclusion

The failure of the Hough Transform in a complex background is mainly caused by irrelevant edges and textures being interpreted as possible circular shapes. Preprocessing reduces unwanted information and improves the quality of the input, while parameter tuning restricts the detection to realistic circles. Therefore, a combination of noise reduction, edge detection, contrast enhancement, ROI selection, and suitable Hough parameters can significantly improve circular shape detection accuracy and reduce false positives.

2. **Feature Matching under Illumination Changes**

   Two images of the same scene are captured under different lighting conditions, causing feature matching errors. Analyze why matching fails and explain how preprocessing or robust descriptors improve performance.

Feature matching attempts to identify corresponding points between two images of the same scene. In this case, matching errors occur because the two images have different illumination conditions, which can change the appearance and intensity of important image features.

Analysis of the Problem

When lighting changes, the same object may appear significantly different in the two images. Bright illumination can increase pixel intensities, while shadows or low illumination can reduce them. These changes affect the local intensity patterns, edges, gradients, and textures used for feature detection and description.

As a result:

- The same feature may have different intensity values in the two images.
- Some features may become less visible under poor lighting.
- Shadows can create additional edges or remove existing ones.
- Descriptors based heavily on raw intensity information may produce different feature representations.
- Incorrect correspondences or missed matches can occur.

Therefore, even though the physical scene is unchanged, its visual representation is different, causing conventional feature matching to become unreliable.

Role of Preprocessing

Preprocessing can reduce the effect of illumination differences before feature extraction and matching.

1. Intensity normalization:

Adjusting image intensity levels makes the brightness distribution of the two images more similar.

2. Contrast enhancement:

Techniques such as histogram equalization or CLAHE can improve feature visibility in dark or unevenly illuminated regions.

3. Grayscale conversion:

Converting images to grayscale removes colour information that may vary with lighting and provides a simpler representation for feature extraction.

4. Illumination correction:

Uneven lighting can be reduced so that the important structural information of the scene becomes more consistent.

These preprocessing techniques help produce more stable features for subsequent matching.

Use of Robust Feature Descriptors

Another solution is to use descriptors that are less sensitive to illumination changes.

For example, **SIFT** uses local gradient information rather than directly depending on individual pixel intensities. This makes its feature representation more robust to changes in brightness and moderate illumination variations.

Other robust descriptors can also represent local structure, edges, or gradients in a way that remains relatively stable when lighting changes.

Comparison

Approach	**Effect under illumination changes**
Raw intensity-based matching	Highly affected by brightness changes
Preprocessing	Reduces differences caused by illumination
Robust descriptors	Preserve more stable local feature information
Preprocessing + robust descriptors	Provides more reliable matching

Best Solution

The most effective approach is to combine illumination preprocessing with robust feature descriptors. Preprocessing reduces the visual differences between the images, while robust descriptors provide feature representations that are less dependent on illumination.

This combination is better than relying only on preprocessing because preprocessing may not completely remove lighting variations. Similarly, using a robust descriptor alone may still be affected when illumination differences are severe.

Conclusion

Feature matching fails under different lighting conditions because illumination changes alter the appearance of the same physical features. Intensity normalization, contrast enhancement, and illumination correction can reduce these variations. Using robust descriptors such as SIFT further improves the stability of feature representation. Therefore, combining suitable preprocessing with illumination-robust descriptors provides more accurate and reliable feature matching between images captured under different lighting conditions.

3. **PCA in Face Recognition**

   A face recognition system processes high-dimensional image data, resulting in slow performance. Analyze how PCA helps in reducing dimensionality while preserving important features.

Principal Component Analysis (PCA) is a dimensionality-reduction technique that transforms a large number of correlated image features into a smaller set of important, uncorrelated features called principal components. In face recognition, this helps reduce the computational cost of processing high-dimensional facial images.

Analysis of the Problem

A facial image may contain thousands of pixel values. When many images are processed, the resulting feature space becomes very large. Processing all these features increases:

- **Memory requirements**
- **Computation time**
- **Processing complexity**
- **Redundant information**

Not every pixel contributes equally to identifying a face. Many pixels may contain similar or less useful information. Therefore, processing the complete feature space is inefficient.

How PCA Solves the Problem

PCA identifies the directions in the data that contain the maximum variance and represents the original data using these principal components.

The main process is:

1. **Prepare the image data**
   Each face image is represented as a numerical feature vector.
2. **Center the data**
   The mean of each feature is calculated and subtracted from the corresponding feature values.
3. **Find principal components**
   PCA determines the directions that capture the greatest variation in the facial data.
4. **Rank the components**
   Components are ordered according to the amount of information or variance they represent.

5. **Select important components**
   Only the most significant components are retained, while components containing relatively little information are discarded.
6. **Transform the images**
   The original high-dimensional images are represented using the selected components.

Why PCA Improves Efficiency

Suppose each face image contains 10,000-pixel features. Instead of using all 10,000 features, PCA may represent the images using a much smaller number of principal components.

This reduces the dimensionality of the data while retaining the major patterns needed for distinguishing faces. Consequently, the face recognition system requires less memory and performs feature comparison and classification faster.

Important Trade-off

PCA should not be applied too aggressively. If too few components are retained, important facial information may be removed, reducing recognition accuracy. Therefore, the number of components should be selected so that most of the important variance is preserved while unnecessary dimensions are eliminated.

Conclusion

PCA improves face recognition efficiency by converting high-dimensional facial image data into a smaller set of meaningful principal components. It reduces computational cost, memory usage, and data redundancy while retaining the major patterns present in the images. However, an appropriate number of components must be selected to maintain sufficient facial information and avoid reducing recognition accuracy.

4. **Edge Detection in Low Contrast Images**

   An image has low contrast, causing edge detection algorithms to fail. Analyze the problem and explain how preprocessing enhances edge detection results.

Edge detection identifies sudden changes in intensity between neighboring pixels to locate boundaries of objects. In a low-contrast image, the intensity difference between an object and its background is very small. Therefore, the gradient at the object boundary may be weak, making it difficult for edge detection algorithms to distinguish actual edges from normal image variations.

Analysis of the Problem

The main problem is the small intensity difference between adjacent regions. Edge detection methods such as Canny depend on sufficient intensity gradients to identify meaningful boundaries. In a low-contrast image:

- Object boundaries may have weak gradients.
- Important edges may not reach the detection threshold.
- Some edges may become fragmented or completely disappear.
- Noise can become relatively significant compared with the weak edges.
- The resulting edge map may contain fewer meaningful boundaries.

Therefore, directly applying edge detection to the raw image can result in incomplete or inaccurate edge detection.

Role of Preprocessing

Preprocessing improves the image before edge detection by making important intensity differences more distinguishable.

1. Contrast Enhancement

Techniques such as **histogram equalization** redistribute image intensities and increase the difference between darker and brighter regions. This makes weak object boundaries more visible.

2. CLAHE

Contrast Limited Adaptive Histogram Equalization (CLAHE) enhances contrast locally rather than applying the same enhancement to the entire image. It is particularly useful when the image contains different lighting conditions in different regions.

3. Noise Reduction

A Gaussian or median filter can reduce unwanted noise before edge detection. This is important because increasing contrast may also make noise more noticeable.

4. Intensity Normalization

Normalizing intensity values provides a more consistent intensity range, helping the edge detector identify meaningful gradients.

Effect on Edge Detection

Without preprocessing, a weak boundary may produce a small intensity gradient that is below the edge detector's threshold. After contrast enhancement, the intensity difference becomes stronger, allowing the edge detector to identify the boundary more reliably.

However, excessive enhancement should be avoided because it can amplify noise and create false edges. Therefore, preprocessing parameters should be selected according to the characteristics of the image.

Suitable Approach

For a low-contrast image, an effective approach is to first reduce noise, then apply contrast enhancement such as CLAHE, and finally perform edge detection using an appropriate threshold. This improves the visibility of weak boundaries while controlling unwanted noise.

Conclusion

Low contrast makes edge detection difficult because the intensity differences between object boundaries and their surroundings are too small. Contrast enhancement, CLAHE, intensity normalization, and noise reduction can strengthen useful image information before edge detection. Proper preprocessing therefore produces clearer, more continuous, and more accurate edges, improving the overall performance of the vision system.

5. **Feature Matching with Noise**

   Feature matching between two images fails due to noise in one image. Analyze the impact of noise and explain how preprocessing improves matching accuracy.

Feature matching identifies corresponding points between two images using distinctive image features and their descriptors. When one of the images contains significant noise, the appearance of these features changes, making it difficult for the matching algorithm to identify the correct correspondences.

Analysis of the Problem

Noise introduces unwanted variations in pixel intensity that are not part of the actual scene. During feature detection, these variations may be interpreted as meaningful image structures.

This can result in:

- False keypoints being detected from noise.
- Changes in the location or strength of genuine features.
- Unstable or incorrect feature descriptors.
- Incorrect correspondences between the two images.
- Fewer correct matches because the noisy image does not produce descriptors similar to those of the clean image.

For example, if a corner or textured region is affected by noise, its local intensity pattern can change significantly. The descriptor generated from that region may therefore differ from the corresponding descriptor in the clean image.

Impact on Feature Matching

Feature matching generally depends on the similarity between descriptors from the two images. If noise changes the local appearance of a feature, the distance between its descriptors increases.

Consequently, the matching algorithm may:

1. Fail to find the correct corresponding feature.
2. Match the feature with an incorrect location.
3. Reject a valid match because the descriptor difference is too large.

Thus, noise reduces both the quality of feature detection and the reliability of feature matching.

Role of Preprocessing

Preprocessing should be performed on the noisy image before feature detection and description.

1. Gaussian Filtering

A Gaussian filter smooths small intensity variations and reduces random noise. This helps prevent noise from being detected as keypoints.

2. Median Filtering

A median filter is particularly useful for removing impulse or salt-and-pepper noise while preserving important edges.

3. Bilateral Filtering

Bilateral filtering reduces noise while preserving significant edges. This can be useful when maintaining feature boundaries is important for subsequent feature detection.

Why Preprocessing Improves Matching

After noise reduction, the image contains fewer unwanted variations. As a result:

- Fewer false keypoints are generated.
- Genuine features become more stable.
- Descriptors become more similar to their corresponding descriptors in the clean image.
- The number of correct matches increases.
- False matches are reduced.

However, excessive smoothing should be avoided because it can remove fine details that are actually useful for feature detection.

Best Approach

The preprocessing method should be selected according to the type and severity of noise. A suitable filter should remove unwanted noise while preserving important edges, corners, and textures. After preprocessing, feature detection and descriptor extraction can be performed before matching.

Conclusion

Noise negatively affects feature matching by introducing false structures and changing the appearance of genuine features. This produces unreliable keypoints and descriptors, leading to incorrect or missed matches. Appropriate noise-reduction preprocessing, such as Gaussian, median, or bilateral filtering, improves matching by preserving meaningful image structures while removing unwanted variations. Therefore, carefully controlled preprocessing can significantly improve feature matching accuracy and reliability.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understandi ng with most issues identified	Basic understanding; some issues missed	Poor understandi ng; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understanda ble presentation	Limited clarity and structure	Poor clarity; difficult to understand